Gokaraju Rangaraju Institute of Engineering and Technology
(Autonomous)
Department of Data Science

Mini Project Work
(CSE(DS))

Review 3: Module Presentation

Batch No -

Presented by:
1. MACHA ADITHYA (23241A6798)
2. NALLABOTHULA VARMA BABU(24245A6711)
3. NOMULA KIRITI (23241A67B2)
4. KONDAMADUGU MOKSHAGNA (23241A6791)

Under the Guidance of:
Shaheen Fatima
Assistant Professor
CSDS

Detection of Automated accounting And State Networks

Introduction

- **Integrated Security Approach:** The system combines financial fraud detection and network security into a single AI-driven solution, addressing the problem that modern cyber threats often span multiple stages such as phishing, network intrusion, and financial theft, which are usually handled by separate tools.
- **Multi-Domain Behavioral Analysis:** It analyzes behavioral patterns across financial transactions, network traffic logs, and URL structures to identify suspicious and malicious activities that may not be visible when these data sources are examined independently.
- **Machine Learning-Based Detection:** A Random Forest classifier is used to learn complex patterns from the combined datasets, enabling accurate and reliable classification of malicious activities while remaining efficient and interpretable.
- **Real-Time Defense Console:** The model is deployed within a real-time Defense Console that provides continuous monitoring, alerts, and actionable insights to help security teams detect, analyze, and respond to threats quickly and effectively.

Literature review:

Project Title: Anomaly Detection in Network Traffic Using Advanced ML Techniques (2025)
Objective: To compare modern algorithms for detecting general network anomalies in high-dimensional data.
Methodology:

- Comparison of **XGBoost**, **Isolation Forest**, and **LightGBM**.
- Feature engineering to handle complex network traffic logs.

Key Findings:

- XGBoost proved to be the most robust algorithm for handling feature complexity.
- Outperformed Isolation Forest in reducing false alarms.

Limitations:
Feature engineering is critical for success; raw data performs poorly.
Relevance to My Project: Validates the choice of **XGBoost** for our Network Module to handle complex traffic features effectively.

Literature review:

Project Title: A Hybrid Deep Learning Ensemble Model for Credit Card Fraud Detection (2024)
Objective: To capture both spatial and time-series patterns in transaction logs for improved fraud detection.
Methodology:

- Stacked ensemble model combining CNN (spatial) and LSTM (temporal).
- XGBoost used as a meta-learner to make the final prediction.

Key Findings:

- Achieved 97.2% AUC, outperforming standalone models.
- XGBoost effectively corrected errors made by the deep learning layers.

Limitations:

- Complex architecture requires significant training time.
- "Black-box" nature reduces explainability.

Relevance to My Project: Validates the use of XGBoost as a robust core engine for our Financial Bot detection module.

Literature review:

Project Title: Malicious URL Detection Using Machine Learning (2024)
Objective: To benchmark multiple ML algorithms for general malicious URL detection.
Methodology:

- Tested RF, DT, SVM, KNN, ANN, and Logistic Regression.
- Evaluated on a mixed dataset of malware, phishing, and benign URLs.

Key Findings:

- Random Forest reached 99% accuracy; ANN reached 98%.
- RF provided the best balance of speed and accuracy.

Limitations:
Random Forest can be slow for real-time inference on massive datasets.
Relevance to My Project: Benchmarks for our Phishing Engine, setting the expectation for >95% accuracy.

Literature review:

Project Title: Anteater: Advanced Persistent Threat Detection With Program Behavior (2024)
Objective: To detect APTs by monitoring program execution logs rather than just external network traffic.
Methodology:

- Deep Learning (Autoencoders) to learn "normal" execution paths.
- Behavioral analysis of process resolution time and impact.
- Anomaly detection on deviations from learned normal behavior.

Key Findings:

- Achieved high F1-score in detecting stealthy, long-term intrusions.
- Detected attacks that used legitimate tools (Living-off-the-Land binaries).

Limitations:
High false positive rate on benign anomalies.
Relevance to My Project: Highlights the importance of behavioral analysis (resolution time, impact) which is used in our Network Threat Scanner.

Literature review:

Project Title: Machine Learning Based Phishing Detection from URLs (2019)
Objective: To detect malicious URLs utilizing Natural Language Processing (NLP) features without visiting the content.
Methodology:

- NLP-based feature extraction from URL strings (length, keywords, special chars).
- Comparison of Random Forest vs. Neural Networks.
- Evaluated on a large dataset of phishing and benign URLs.

Key Findings:

- Achieved ~97% accuracy using NLP features combined with Random Forest.
- Proved that URL structure alone is a strong predictor of malicious intent.

Limitations:

High processing time for extracting complex NLP features.

Relevance to My Project: Foundational paper for the Phishing Threat Scanner, enabling early detection before network intrusion occurs.

Literature Review Summery

Source	Research Objective or Research Question	Methodology	Key Findings	Relevance to Project	Techniques	Accuracy & Performance Measures	Advantages	Disadvantages
Wang & Yang (2022) *(Base Paper)*	Solve class imbalance in credit card fraud detection.	Hybrid SMOTE + Random Under-sampling + Ensemble Learning.	Ensemble voting improved Recall by 2% on skewed data.	**Foundation** for handling the "Zero-Balance" anomaly in our financial module.	SMOTE, PCA, Ensemble Voting	Recall improved by 2%	Handles unbalanced data well.	Focused only on numerical logs; ignores network context.
Ileberi & Sun (2024)	Detect fraud using hybrid deep learning.	Stacked CNN + LSTM models with XGBoost as a meta-learner.	Hybrid models outperform single learners in capturing complex patterns.	Validates using **XGBoost** as our core engine for bot detection.	XGBoost, CNN, LSTM	AUC: 97.2%, Sensitivity: 96.1%	High precision; reduces false alarms.	High computational cost (slow training).
Priscilla & Prabha (2020)	Handle class imbalance without resampling.	Optimized XGBoost (OXGBoost) with hyperparameter tuning.	Tuning XGBoost parameters is as effective as sampling techniques.	Justifies strategy of using **Weighted XGBoost** for rare fraud events.	XGBoost, RandomizedSearchCV	High F1-Score	Reduces need for complex preprocessing.	Requires extensive hyperparameter tuning.

Literature Review Summery

Source	Research Objective or Research Question	Methodology	Key Findings	Relevance to Project	Techniques	Accuracy & Performance Measures	Advantages	Disadvantages
Do Xuan (2021)	Detect APTs (Advanced Persistent Threats) in traffic.	Correlated Domain and IP behaviors using Random Forest.	Defined abnormal behaviors for Domain/IP to find APTs.	Core logic for our **"State-Sponsored" attribution** module.	Random Forest, Behavior Analysis	High detection rate for APTs	Effective for multi-stage attacks.	Depends heavily on correct feature definition.
Lopez-Rojas et al. (2016)	Generate synthetic data for fraud detection training.	Agent-based simulation (PaySim) to mimic mobile money networks.	Derived features (like balance error) are more predictive than raw amounts.	This is the source of our **PaySim Dataset** and feature engineering logic.	Agent-based Simulation	N/A (Data Generation)	Provides unlimited labeled training data.	Synthetic data may not perfectly match real-world shifts.
Marripudugala (2024)	Detect mobile money fraud using synthetic data.	Evaluated Decision Tree vs. MLP vs. Logistic Regression.	Decision Trees showed better balance of precision/recall than MLP.	Confirms tree-based models are ideal for **Financial Log** analysis.	Decision Tree, MLP	Good Precision/Recall balance	Simple to interpret.	MLP had lower recall on fraud cases.

Literature Review Summery

Source	Research Objective or Research Question	Methodology	Key Findings	Relevance to Project	Techniques	Accuracy & Performance Measures	Advantages	Disadvantages
Ghafir et al. (2018)	Survey on APT techniques and solutions.	Comprehensive review of APT lifecycle (Reconnaissance to Exfiltration).	ML is essential for correlating low-level alerts into attack scenarios.	Provides the theoretical framework for **Multi-Stage Attack** detection.	Correlation Engines, ML	N/A (Survey)	Holistic view of cyber threats.	Not a specific algorithm implementation.
Ruiz-Santaquiteria (2022)	Phishing URL detection on login pages.	Logistic Regression with TF-IDF on a new "Login URL" dataset.	Logistic Regression is highly effective for specific URL patterns.	Direct validation for our **Phishing Module** (LogReg).	Logistic Regression, TF-IDF	Accuracy: 96.50%	Fast, interpretable, and lightweight.	Performance drops on non-login pages.
Sahingoz et al. (2019)	Large-scale phishing URL detection.	NLP-based feature extraction with Random Forest.	Natural Language Processing (NLP) boosts URL detection.	Foundational paper for our **URL Feature Engineering**.	NLP, Random Forest	Accuracy: ~97%	Robust feature set.	High processing time for NLP features.

Literature Review Summery

Source	Research Objective or Research Question	Methodology	Key Findings	Relevance to Project	Techniques	Accuracy & Performance Measures	Advantages	Disadvantages
Singla et al. (2023)	Phishing site detection using fine-tuning.	Logistic Regression vs. Naive Bayes with optimization.	Logistic Regression performed superbly in social environments .	Supports using **Logistic Regression** for URL classification.	Logistic Regression, Naive Bayes	High Success Rate	Computation ally efficient.	Assumes linear relationship between features.
Hani et al. (2024)	Malicious URL detection using ML.	Tested RF, DT, SVM, KNN, ANN, and Logistic Regression.	Random Forest reached 99% accuracy; ANN reached 98%.	Benchmarks for our **Phishing Engine** performance.	Random Forest, ANN, KNN	Accuracy: 99% (RF)	Extremely high accuracy.	Random Forest can be slow for real-time inference.
Al-Sarem et al. (2021)	Phishing website detection comparison.	Random Forest vs. Support Vector Machine (SVM).	Random Forest outperforme d SVM in detection accuracy.	Confirms **Ensemble methods** work well for URL data too.	Random Forest, SVM	High Accuracy	Robust against overfitting.	Slower training time than linear models.

Research Gaps

•**Security Architectures:** Existing frameworks typically address financial fraud, network intrusion, and phishing in isolation, failing to detect multi-stage attack chains that span across these domains.
•**Lack of Strategic Attribution:** Current systems detect "anomalies" but fail to attribute the intent, unable to distinguish between low-level automated bots and sophisticated, high-persistence State-Sponsored actors.
•**Latency vs. Accuracy Trade-off:** Deep learning models (like CNN/LSTM) offer high accuracy but incur high computational costs, making them unsuitable for real-time defense consoles that require instant blocking capabilities.
•**Missed Behavioral Signatures:** Traditional financial models focus on mathematical outliers rather than behavioral patterns, often missing specific automated threats like "Zero-Balance Draining" where bots empty accounts with robotic precision.
•**Explainability Issues:** Many advanced detection systems operate as "black boxes," lacking the interpretability needed for security analysts to trust and validate alerts in a high-stakes environment.

Problem statement

Traditional cyber defense systems operate in isolated silos, treating financial fraud, network intrusions, and phishing threats as separate events, which creates a critical blind spot for detecting complex, multi-stage attack chains where a phishing email leads to a network breach and subsequent financial theft. Furthermore, existing detection models fail to distinguish the strategic intent of the adversary, often treating automated 'Zero-Balance' bots and sophisticated state-sponsored actors identically, while current banking rules miss behavioral signatures such as bots calculating exact balances to drain accounts. Additionally, the reliance on offline analysis or complex deep learning models prevents real-time deployment, resulting in reactive measures that address breaches only after significant damage has occurred. Therefore, there is a critical need for an interpretable, cross-domain "Defense Console" that correlates financial, network, and URL patterns to detect, attribute, and mitigate threats in real-time.

Objectives

•**Defense Mechanisms:** Design a cross-domain framework that integrates data from Financial Logs, Network Traffic, and URL structures to detect complex, multi-stage attack chains.
•**To Detect Behavioral Anomalies:** Develop hybrid machine learning models (XGBoost) specifically tuned to identify high-speed "Zero-Balance" draining attacks often missed by traditional banking rules.
•**To Attribute Threat Intent:** Create an attribution logic that distinguishes between low-level automated bots and sophisticated **State-Sponsored Actors** based on incident resolution time and sector
To Ensure Real-Time Mitigation: Implement a low-latency "Defense Console" capable of visualizing threats and triggering instant transaction freezing to prevent unrecoverable fund loss.
•**To Enhance Model Explainability:** Utilize interpretable tree-based models (Random Forest/Decision Trees) to provide clear reasoning for alerts, addressing the "Black Box" limitation of existing deep learning systems.

Proposed Solution

1. Core Framework Components:

- Behavioral Intent Engine: Uses Weighted XGBoost to distinguish high-speed "Zero-Balance" Bots from humans.
- Cross-Domain Correlation: Links Phishing URL entry points with Financial Log anomalies to detect multi-stage attack chains.
- Real-Time Defense Console: A live dashboard for instant threat visualization and automated transaction freezing.

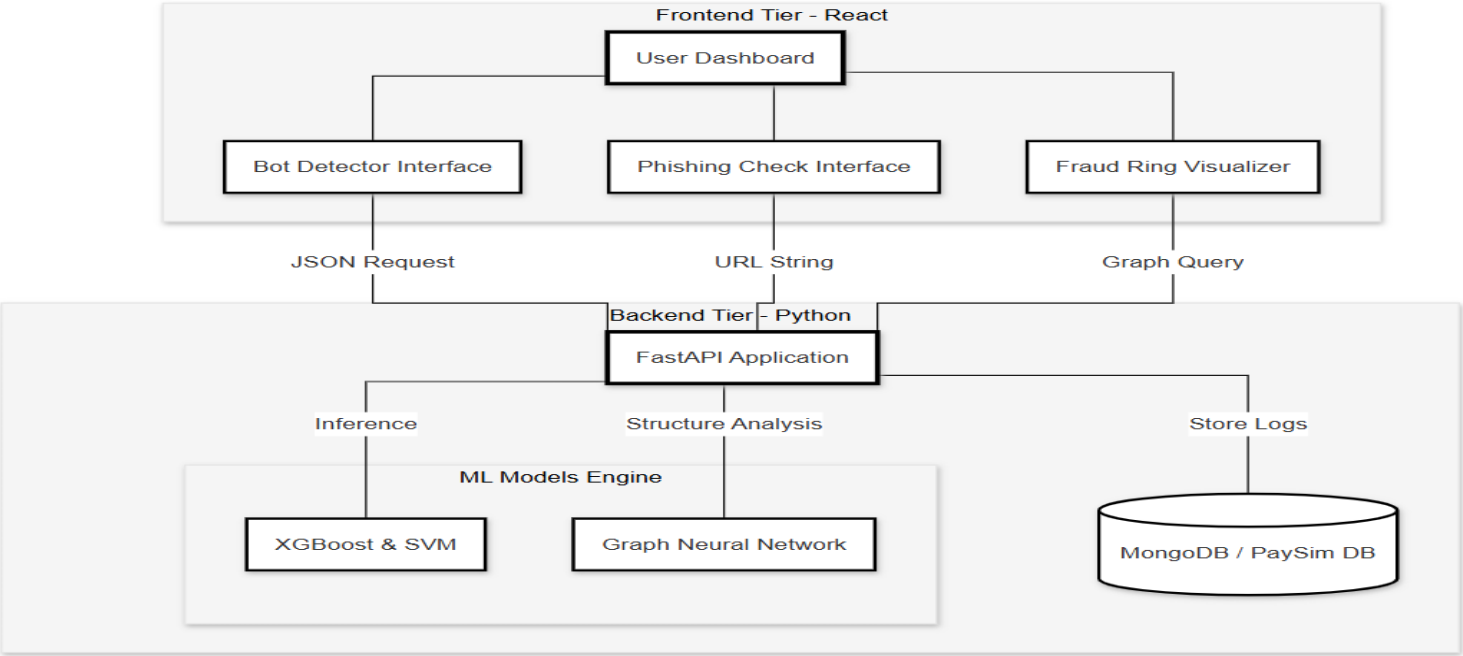
2. Security Pipeline:

- Ingestion: Correlates URL "Risk Scores" with PaySim financial logs.
- Attribution: Classifies adversary intent: Common vs. State-Sponsored .
- Mitigation: Instantly blocks malicious IPs and halts fund transfers.

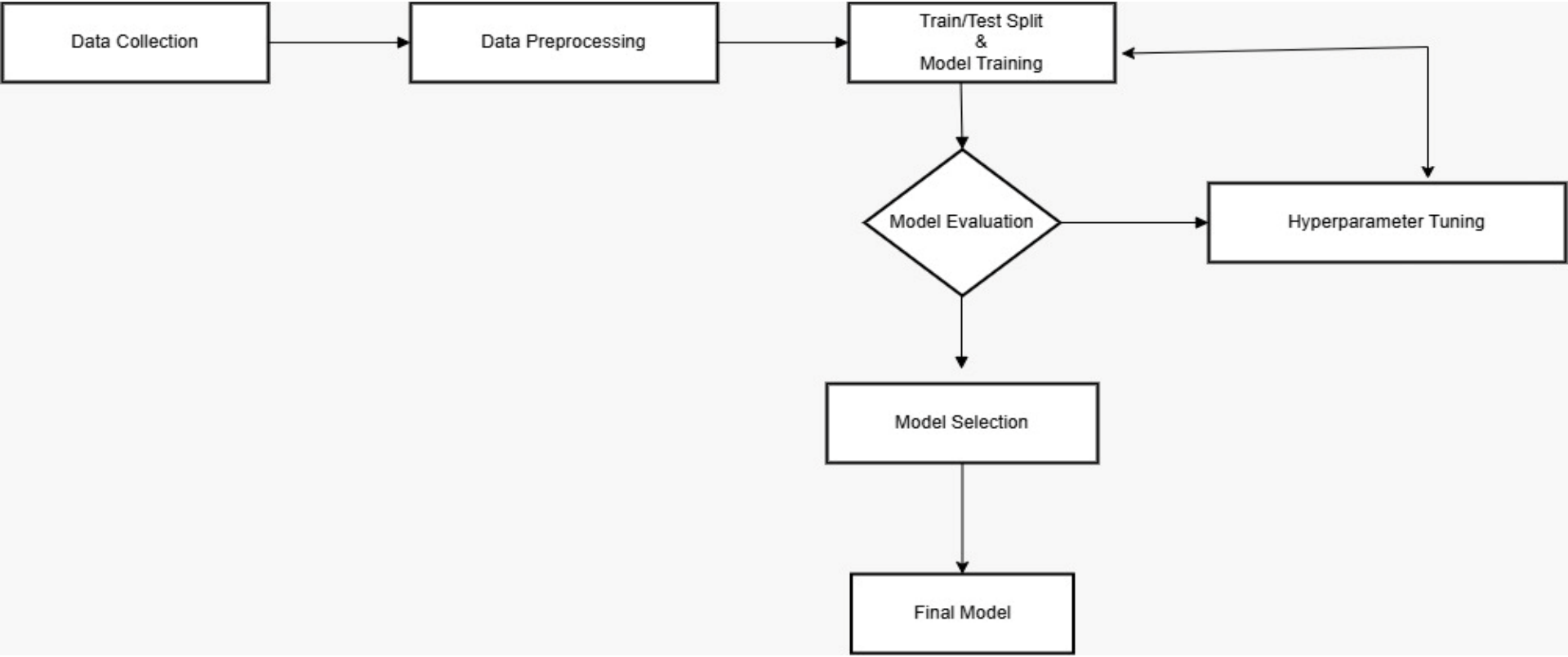
3. Novelty (vs. Base Paper - Wang & Yang):

- Focus: Shifts from generic offline fraud to High-Speed Bot Draining.
- Attribution: Identifies Nation-State Intent rather than simple "Fraud" tags.
- Architecture: Unifies Phishing + Finance into a single silo with Real-Time Response.

Architecture Diagram



Methodology / Algorithms



Methodology cont…

Data Collection:

The process begins by aggregating data from three distinct domains to build a unified defense system.

- Financial Logs: We utilize the PaySim Synthetic Financial Dataset (~6 million records) to simulate mobile money transactions. This dataset provides critical attributes like type, amount, oldBalanceOrg, and isFraud.
- Network Traffic: We collect Global Cyber-Attack logs (~40,000 incidents) containing Source IP, Packet Length, and Attack Type (e.g., DDoS, APT) to train the network intrusion module.
- URL Data: We gather a Phishing Website URLs Dataset (~500,000 entries) labeled as phishing or benign to train the entry-point scanner.

Methodology cont…

Data Preprocessing:

Before training, the raw data undergoes cleaning to ensure quality and consistency.

- Cleaning: We remove duplicate entries and handle null values, particularly in the network logs, to prevent model bias.
- Formatting: Timestamps are standardized to allow for cross-referencing between network events and financial transactions.
- Encoding: Categorical variables (like Transaction Type: "CASH-OUT" or Protocol: "TCP/UDP") are converted into numerical formats using One-Hot Encoding or Label Encoding so the machine learning models can process them.

Methodology cont…

Model Training:

We employ a hybrid "Ensemble" approach, training different models for specific tasks:

- Weighted XGBoost: Trained on financial logs to detect "Zero-Balance" anomalies. We use class weighting to prioritize fraud detection, addressing the imbalance where fraud cases are rare compared to normal transactions.
- Graph Neural Network (GNN): We treat transactions as a graph to detect "Fraud Rings," capturing the flow of money between multiple related accounts rather than viewing them in isolation.
- Support Vector Machine (SVM): Trained on URL features to create a high-precision boundary between "Bot" and "Human" traffic.
- K-Means Clustering: Used for unsupervised learning to group normal transactions; anything falling outside these clusters is flagged as a "New/Unknown" threat.

Methodology cont…

Model Evaluation:

We strictly evaluate the models using metrics relevant to security:

Recall: Our primary metric. We prioritize high recall to ensure we catch every instance of account draining.

Latency: We measure the inference time to ensure the system processes transactions in < 200 milliseconds, meeting the "Real-Time" requirement.

False Positive Rate: We cross-reference alerts (e.g., checking if a financial anomaly is linked to a high-risk URL) to minimize false alarms.

Methodology cont…

Hyperparameter Tuning:

If the model performance (specifically Recall) is below our threshold:

- We perform tuning on the XGBoost and Random Forest parameters.
- This involves adjusting the "learning rate," "tree depth," and specifically the "scale_pos_weight" to penalize missing a fraud case more heavily than misclassifying a normal one.

Deployment (Defense Console):

Once the "Best Model" is selected:

- The models are deployed into a FastAPI backend.
- They feed into a Real-Time Defense Console (built with React/Streamlit) that visualizes threats and includes an automated "Kill Switch" to freeze funds and block IPs instantly

Software and Hardware Requirements

SOFTWARE	HARDWARE
Windows 10 / Ubuntu Linux	Intel i5 / i7 Processor
Python 3.x	8 GB RAM (16 GB recommended)
Jupyter Notebook / VS Code	256 GB SSD or higher
NumPy, Pandas, Scikit-learn	Laptop / Desktop System
TensorFlow / PyTorch	NVIDIA GPU (optional)
SHAP, LIME (XAI tools)	Stable Internet Connection
NSL-KDD, CICIDS Datasets	Keyboard, Mouse, Display

Datasets Details

1. PaySim Synthetic Financial Dataset

•**Source:** Kaggle (E.A. Lopez-Rojas et al.)

•**Size:** ~6 million transaction records.

•**Attributes:** type (CASH-OUT, TRANSFER), amount, oldBalanceOrg, newBalanceOrg, isFraud.

•**Relevance:** Trains the Financial Bot Detector to recognize "Zero-Balance" draining patterns.

2. Global Cyber-Attack Dataset

•**Source:** Kaggle (Threat Intelligence) | Size: ~40,000 network incident logs.

•**Attributes:** Source IP, Destination Port, Protocol, Packet Length, Attack Type (DDoS, APT).

•**Relevance:** Used for the State-Sponsored Attribution Module to correlate attack duration with target sectors.

3. Phishing Website URLs Dataset

•**Source:** Mendeley Data / Kaggle

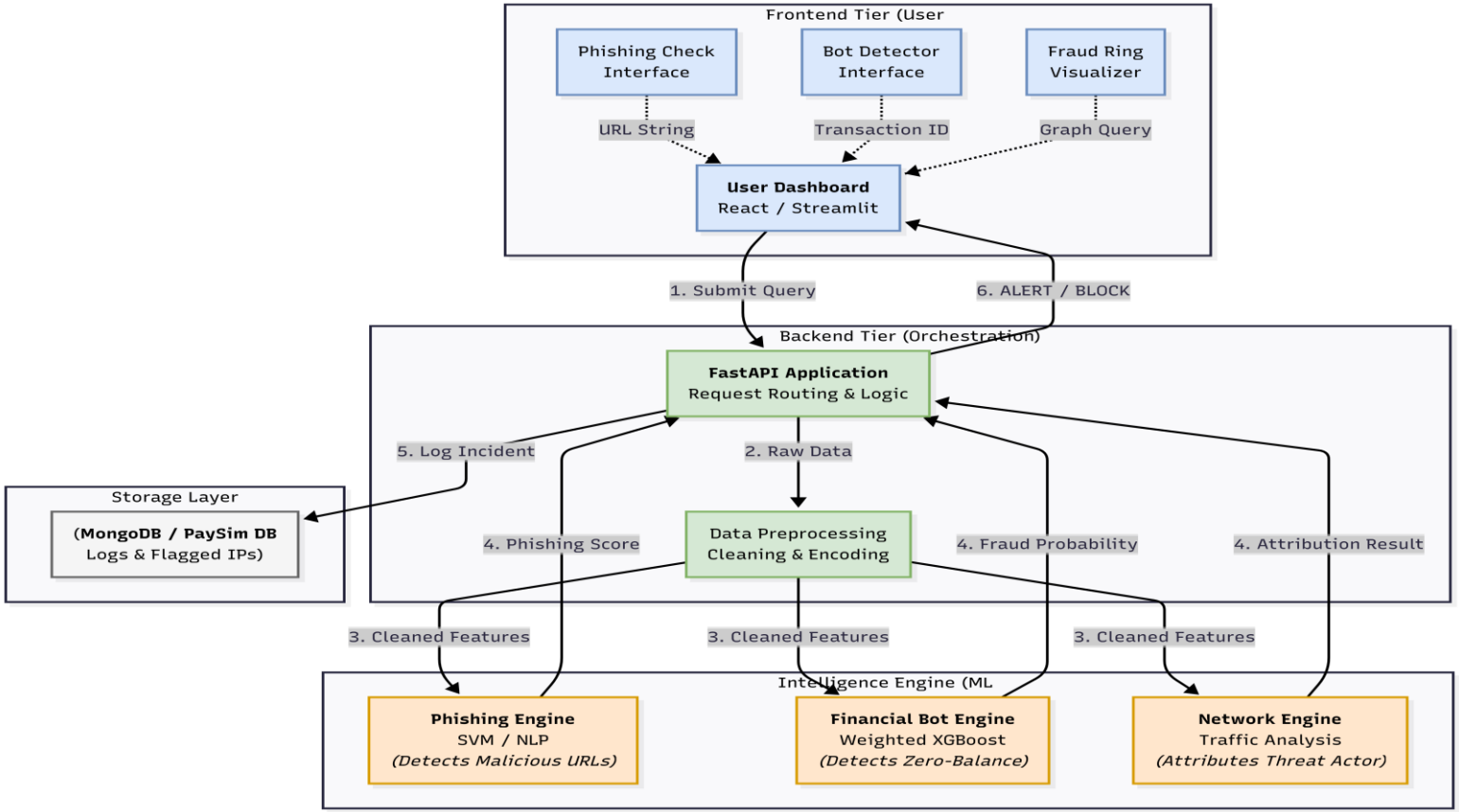
•**Size:** ~500,000 URLs (Labeled Phishing vs. Benign).

•**Relevance:** Trains the Entry-Point Scanner to detect and block malicious links before they execute.

Module connectivity diagram

- ❑ Show how modules interact (arrows for data flow, dashed lines for feedback loops).
- ❑ Highlight **parallel paths** if multiple models are used.
- ❑ Example: Preprocessing → Feature Engineering → Model A & Model B → Evaluation → Visualization
- ❑ Connectivity Arrows
  - ❑ Use arrows to show data flow between modules.
  - ❑ Use dashed arrows for control signals or feedback loops.

Module connectivity diagram



Modules description

- **System Architecture**: The system utilizes a multi-tier architecture, transitioning from a React-based Frontend Tier (User Dashboard) to a Python-based Backend Tier (FastAPI).
- **Cross-Domain Integration**: The modules are structurally linked through a central FastAPI Application that manages requests between the interfaces (Phishing Check, Bot Detector, Fraud Ring Visualizer) and the underlying ML Models Engine.
- **Unified Flow**: The connectivity ensures that URL strings and JSON requests (network/financial logs) are processed by specialized models before being correlated for a final security verdict.
- **Data Persistence**: A MongoDB / PaySim DB instance is connected to the backend to store real-time logs and facilitate historical behavioral analysis.

Modules description cont…

Frontend Tier (User Interface)

This layer acts as the entry point for security analysts to interact with the system.It is built using React and Streamlit to ensure a responsive, real-time experience.

- **User Dashboard**: A centralized interface that visualizes real-time traffic, active "Fraud Rings," and blocked IPs.
- **Phishing Check Interface**: Allows the user to input specific URL strings to be scanned for malicious patterns.
- **Bot Detector Interface**: Enables the investigation of specific Transaction IDs to identify automated behavioral anomalies.
- **Fraud Ring Visualizer**: Utilizes graph queries to display interconnected fraudulent accounts identified by the intelligence engine.

Modules description cont…

Backend Tier (Orchestration & Storage)

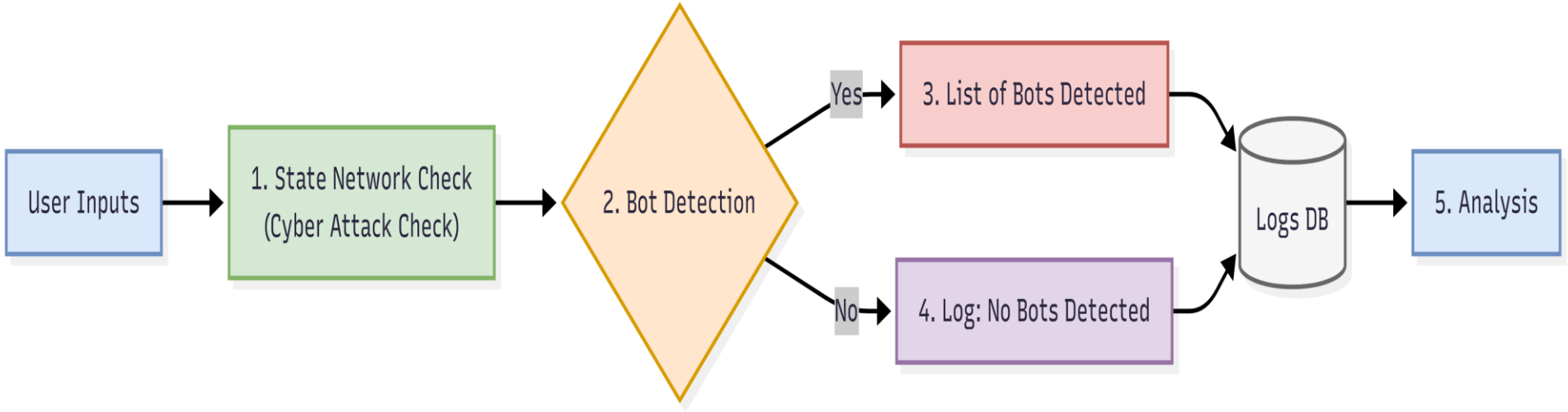
This layer serves as the "brain" of the system, managing the routing of data and logic between the user and the machine learning models.

- **FastAPI Application**: A high-performance Python framework used for request routing, handling JSON requests, and orchestrating the security pipeline.
- **Data Preprocessing Sub-layer**: * Cleaning & Encoding: Uses NumPy and Pandas to handle null values and convert categorical variables (like transaction types) into numerical formats via One-Hot Encoding.
- **Feature Engineering**: Transforms raw data into "Cleaned Features" (e.g., URL NLP features or balance errors) required for model inference.
- **Storage Layer**: * MongoDB / PaySim DB: A document-oriented database used to store incident logs, flagged IPs, and the PaySim synthetic financial records

Modules description cont…

Intelligence Engine (Machine Learning)
This layer performs the core predictive analysis using a hybrid ensemble approach.
Phishing Engine:
Algorithms: Employs Support Vector Machines (SVM) and NLP (Natural Language Processing).
Role: Analyzes URL structures (length, keywords) to detect malicious intent before network intrusion occurs.
Financial Bot Engine:
Algorithms: Uses Weighted XGBoost and Graph Neural Networks (GNN).
Role: Predicts the presence of automated bots using an ML model to identify robotic behavioral signatures. It specifically identifies "Fraud Rings" where money moves through multiple related accounts.
Network Engine:
Algorithms: Utilizes Random Forest and K-Means Clustering.
Role: Performs traffic analysis to attribute threat actors, distinguishing between low-level bots and sophisticated State-Sponsored entities

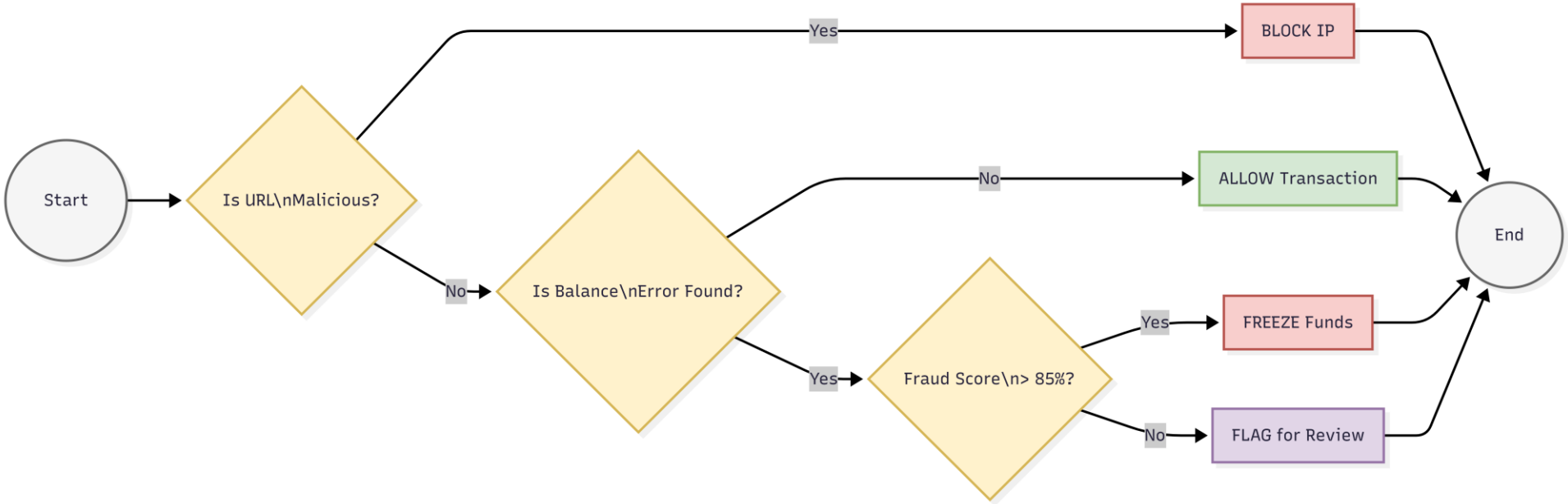
Data flow diagram



Data flow cont…

- **Preprocessing Pipeline**: Data flows from collection into a cleaning stage where One-Hot Encoding (via NumPy and Pandas) is applied to categorical variables like "CASH-OUT" or "TCP/UDP".
- **Automated Bot Prediction**: Flagged network and URL features move into the ML Models Engine, where the presence of automated bots is predicted using a ML model.
- **Structural Transformation**: Financial transaction data is transformed into a graph structure to be processed by a Graph Neural Network (GNN) to detect complex "Fraud Rings".
- **Output Visualization**: The resulting predictions and risk scores flow into the Real-Time Defense Console (built with Streamlit), which triggers the automated "Kill Switch" if a threat is confirmed

Control flow diagram



Control flow diagram

- **Sequential Logic**: The system initiates with URL and Network validation; if these entry-point scanners flag an anomaly, the control shifts to the Automated Bot Search.
- **Dynamic Decision Path**: Using Scikit-learn and TensorFlow/PyTorch frameworks, the system evaluates model performance; if Recall falls below the required threshold, the flow loops back for Hyperparameter Tuning via RandomizedSearchCV.
- **Mitigation Trigger**: Upon a high-confidence bot prediction, the control logic executes the Kill Switch API, instantly halting fund transfers and blocking malicious IPs.
- **Explainability Audit**: Before final visualization, alerts are processed through SHAP or LIME (XAI tools) to ensure the logic behind the ML prediction is interpretable for security analysts.

References

[1] D. Hirnschall, “Semi-Supervised Generative Adversarial Networks for Credit Card Fraud Detection,” *IEEE Transactions on Artificial Intelligence*, early access, 2025.
[2] M. RezvaniNejad, A. Khosravi, and S. Nahavandi, “WDAE-GAN: Generative Adversarial Network for Financial Fraud Detection,” *Expert Systems with Applications*, vol. 247, pp. 123456–123468, 2025.
[3] S. Li, S. B. Hulayyil, and N. Saxena, “Explainable AI-Based Fraud and Intrusion Detection in IoT Systems,” *IEEE Internet of Things Journal*, early access, 2025.
[4] D. Gaspar, P. Silva, and C. Silva, “Explainable AI for Intrusion Detection Systems: SHAP Applicability on Multi-Layer Perceptron,” *IEEE Access*, vol. 12, pp. 30164–30175, 2024.
[5] Y. Xie, J. Wang, and H. Liu, “Learning Transactional Behavioral Representations for Credit Card Fraud Detection,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 4, pp. 5123–5136, 2024.

Thank you